

프로젝트 진행 보고서

[작성자]

이름 : 최범수

휴대폰 : 010-9110-1877

이메일 : qjaqjaqja1877@gmail.com

[기본 정보]

과정명	GCC 사관학교 게임콘텐츠 과정
파트	개인 미니 프로젝트

[프로젝트 개요]

제목	Neon Drift
한 줄 설명	호버바이크로 자원을 모으고 사방에서 밀려오는 적을 막는 아케이드 디펜스 게임. 언리얼 엔진 5 위에서 블루프린트 없이 C++만으로 3일 안에 완성했고, 입력·UI·물리·상태머신까지 게임플레이 전체를 직접 구현.
배경 또는 기획 의도	언리얼 엔진을 쓰면서 항상 블루프린트에 의존해왔다. 한 번쯤은 에디터의 도움 없이 C++만으로 게임 하나를 처음부터 끝까지 만들어 보고 싶어 시작하게 되었다. 아케이드 감성의 게임을 좋아하는데, 화려한 네온 비주얼과 빠른 조작감을 C++로 직접 구현해 보고 싶었다. 또한 AI 도구를 최대한으로 사용해가면서 협업과 리뷰가 가능한 코드베이스를 만드는 실험을 하고 싶었다.
참여 인원	1인 개발

본인 역할	1인 개발 — 아키텍처 설계 및 C++ 구현 / AI 코딩 도구 활용
진행 기간	2026.05.19 ~ 2026.05.22

[관련 자료]

자료	내용	링크 / 위치
게임 기획서	코어 루프 설계, 웨이브 밸런스, 강화 수치 정의	https://zrr.kr/2J3dU6
구현 설계서	클래스 다이어그램, 상태머신, 데이터 테이블 정의	https://zrr.kr/LsCOVj
GitHub 저장소	소스 코드 전체	https://github.com/iamfreakin/NEON-DRIFT
브랜딩 랜딩 페이지	GIF·기술 스택·문서 링크	https://iamfreakin.github.io/NEON-DRIFT/
클래스 다이어그램	전체 클래스 구조 및 의존 관계	별첨 이미지
플로우차트 (3종)	게임 루프·자원 채집·전투 처리 흐름	별첨 이미지

[기술 스택]

스택	선택 이유
UE 5.8	상용 엔진 경험 + 검증된 렌더링 기반 AActor, APawn, AGameMode, UGameInstance 등 계층 구조 활용
C++20	블루프린트 0%, 버전 관리 가능한 코드베이스
Niagara	리본 렌더러로 기체·몬스터 이동 트레일 구현 속도감과 궤적을 시각적으로 표현하는 가장 간단한 방법

[프로젝트 진행 단계]

1. 기획 & 설계

- 게임 컨셉 확정, 코어 루프 정의, 클래스 다이어그램 작성, 상태머신 7단계 설계

2. 프로젝트 기반 구축

- UE5 블랭크 템플릿 세팅, 빌드 환경 구성, 기본 클래스 골격 작성, IDamageable 인터페이스 정의

3. 핵심 메커닉 구현

- 호버바이크 6DOF 비행 물리, Enhanced Input 런타임 생성, 발사 및 라인트레이스, 자원 블록 & 자석 시스템

4. 전투 시스템 구현

- 몬스터 스폰 & 웨이브 펌프, 수동 터렛 탑승, 자동 터렛, 기지 HP 및 게임오버 조건

5. UI & 게임 루프 완성

- Canvas HUD, UMG 상점 화면, GameInstance 업그레이드 영속화, 상태머신 전이 완성

6. 폴리시 & 마무리

- Dynamic Material Instance 피격 플래시 & 네온 색상, Niagara 트레일, 속도 비례 FOV, 이중 크로스헤어, 밸런스 튜닝

[프로젝트 세부 과정]

1단계 - 기획 & 설계

코딩을 시작하기 전에 클래스 다이어그램과 상태머신을 완전히 확정했다. 3일이라는 기간 안에 완성할 수 있었던 가장 큰 이유는 구현 도중에 구조를 바꾸는 일이 없었기 때문이다.

- 코어 루프를 Gather → Combat → Shop 세 단계로 확정
- 게임 상태를 7단계로 정의하고 전이 조건을 모두 문서화한 뒤 코딩 시작
- 클래스 책임을 GameMode(흐름) / GameInstance(영속 상태) / PlayerController(입력)으로 명확히 분리
- IDamageable 인터페이스를 먼저 설계해 이후 피격 로직이 타입 분기 없이 통일되도록 사전 결정

설계에 시간을 충분히 투자한 덕분에 구현 단계에서 방향 변경 없이 진행할 수 있었다. 반대로 말하면, 설계 없이 시작했다면 3일 안에 완성하지 못했을 것이다.

코어 루프 상세 규칙과 밸런스 수치는 **게임 기획서(별첨)**에, 클래스 책임 분리와 인터페이스 설계 근거는 **구현 설계서(별첨)**에 정리했다.

2단계 - 프로젝트 기반 구축

UE5 블랭크 C++ 템플릿에서 시작. 에디터 에셋 의존 없이 CLI 빌드만으로 검증 가능한 구조를 처음부터 목표로 설정했다.

- Build.cs에 필요한 모듈만 등록. 과도한 의존성 추가 없이 필요한 시점에 추가하는 방식 유지
- IDamageable 인터페이스를 가장 먼저 작성해 이후 모든 피격 액터가 이 계약을 따르도록 강제
- GameMode에 Phase enum과 전이 함수 골격만 먼저 작성. 구현은 단계별로 채움

IDamageable 인터페이스 설계 의도와 클래스 간 의존 방향은 아래 **클래스 다이어그램**을 참고.

3단계 - 핵심 메커닉 구현

호버바이크 비행감이 게임 전체의 완성도를 좌우한다고 판단해 가장 먼저 구현했다. 이 단계에서 시간이 가장 많이 걸렸고, 그만큼 가장 많이 다듬은 부분이기도 하다.

- 비행 물리 — CharacterMovementComponent를 쓰지 않고 매 틱 Velocity를 직접 적분했다. 이 선택은 6DOF 이동과 중력 없는 호버를 자유롭게 제어하기 위해서였고, 그만큼 물리 감쇠를 직접 책임져야 했다
- 프레임 독립 감쇠 — $Velocity * = 0.95f$ 방식이 프레임레이트에 의존한다는 것을 테스트 중 발견하고 $FMath::Pow(DragCoeff, DeltaTime)$ 으로 수정 (트러블슈팅 2 참고)
- Enhanced Input 런타임 생성 — .uasset 없이 코드로만 입력 시스템을 구성했다. 이 방식은 UE 공식 지원 경로이지만 초기화 타이밍에 주의가 필요하다 (트러블슈팅 1 참고)
- 자원 블록 공격력 게이트 — IDamageable의 AttackerPower 매개변수를 통해 플레이어 공격력이 RequiredPower에 못 미치면 데미지가 0이 되도록 설계. 업그레이드 동기를 별도 UI 없이 자연스럽게 유도하는 구조

4단계 - 전투 시스템 구현

GameMode가 웨이브 흐름 전체를 소유하고, 나머지 액터들은 GameMode의 Phase를 참조해 자신의 동작을 결정하는 구조로 일관되게 구현했다.

- 웨이브 펌프 — 스폰 포인트 액터를 BeginPlay 때 TActorIterator로 수집. 웨이브 정의에 따라 활성화 방향을 늘려가는 방식. 새 스폰 방향 추가 시 GameMode의 웨이브 데이터만 수정하면 되는 구조
- 수동 터렛 탑승 — SetViewTargetWithBlend로 카메라 전환, 호버바이크 Velocity 잠금, 입력 라우팅 변경을 하나의 탑승 함수에서 처리
- 이중 크로스헤어 — 포신 회전 속도 제한을 두어 카메라와 포신 사이에 의도적인 지연을 만들었다. 이 지연이 시각적 피드백이 되어 조준에 무게감을 줌
- 자동 터렛 — 현재 구현은 직진 단일 패턴 몬스터를 상정한 단순한 탐색·사격 루프다. 몬스터 AI가 복잡해지면 탐색 로직도 개선이 필요한 부분

5단계 - UI & 게임 루프 완성

UI 전체를 C++ 코드로만 구성한 것은 "위젯 블루프린트도 바이너리 에셋"이라는 원칙 때문이었다. 다만 에디터 미리보기 없이 좌표를 잡아야 하는 비용이 크다는 점은 인정한다. 상용 프로젝트라면 핵심 로직만 코드로 두고 배치는 에디터에서 하는 것이 더 현실적이다.

- Canvas HUD — 매 프레임 좌표 계산이 필요한 요소(이중 크로스헤어, HP바)는 AHUD::DrawHUD()에서 처리
- UMG 상점 — 페이지별로 들고 나는 UI는 UUserWidget으로 분리. 위치·크기는 UCanvasPanelSlot으로 코드에서 직접 지정
- GameInstance 영속화 — 강화 상태와 파생 스탯 계산을 GameInstance 한 곳에서 책임. 웨이브 전환·재시작 후에도 상태가 유지되는 단일 진실 공급원 역할

6단계 - 폴리시 & 마무리

게임이 동작하는 상태에서 시각·청각 피드백을 추가했다. Niagara는 리본 렌더러로 기체·몬스터 트레일만 구현했으며, Niagara 전체 기능을 다 활용했다고 말하기는 어렵다.

- Dynamic Material Instance — 부모 머티리얼 1개에서 MID를 생성해 피격 플래시·네온 색·HP 경고색을 Tick에서 파라미터로 제어
- Niagara 리본 렌더러 — 이동 궤적 시각화에 한정해 사용. 에셋 1개, 파라미터만 C++에서 제어
- 밸런스 튜닝 — 물리 파라미터(DragCoeff, 자석 반경, 웨이브 간격)를 PIE 반복 테스트로 조정. 수치보다 체감 기준으로 결정

[트러블 슈팅]

1. Enhanced Input 초기화 타이밍 오류

- ❖ BeginPlay()에서 AddMappingContext를 호출했는데 입력이 반응하지 않았다. 에러 로그도 없어서 원인 파악에 시간이 걸렸다.
- BeginPlay 시점에 UEnhancedInputLocalPlayerSubsystem이 초기화되지 않은 경우가 있다. Subsystem이 null인 상태에서 호출이 조용히 실패하고 있었다.
- AcknowledgedPawn 콜백 시점으로 이동. PlayerController가 Pawn을 완전히 소유한 이후라 Subsystem 초기화가 보장된다.

UE 초기화 순서 문서를 먼저 읽었다면 막히지 않았을 문제다. 런타임 생성 방식은 공식 지원 경로지만 초기화 순서에 민감하기 때문에 다음부터 신경을 써야 할 것 같다.

2. 프레임레이트 의존 물리 감쇠

- ❖ $Velocity *= 0.95f$ 로 구현했더니 기기마다 제동 거리가 달랐다.
- $0.95f$ 는 프레임당 고정 감쇠율이라 프레임 수가 많을수록 감쇠가 더 많이 누적된다.
- $FMath::Pow(DragCoeff, DeltaTime)$ 으로 변경. 단위 시간당 감쇠율이 프레임 수와 무관하게 일정해진다.

게임 물리 기초 지식이 있었다면 처음부터 이렇게 짤 것이다. 이 문제를 직접 겪고 나서 프레임 독립 연산이 왜 중요한지 실감했다.

3. UMG C++ 좌표 계산 비용

- ❖ 에디터 미리보기 없이 모든 UI 위치를 코드로 계산하다 보니 빌드-실행-확인 사이클이 반복됐다.
- 앵커(비율 좌표)와 오프셋(픽셀 좌표) 두 좌표계가 혼재하는데, 이를 정확히 이해하지 못한 채 수치를 추정했다.
- 앵커·오프셋 체계를 정리하고 공통 레이아웃 상수를 헤더에 추출. 이후 수정 속도가 빨라졌다.

이 방식은 Git diff로 UI 변경이 추적된다는 장점이 있지만, 초기 비용이 크고 디자인 변경이 잦은 환경에서는 맞지 않는다. 이번 프로젝트의 "코드 100%" 원칙을 지키기 위한 의도적

트레이드오프였지, 모든 상황에서 권장할 방식은 아니다.

4. 호버바이크 비행감 튜닝

- ❖ 물리 수치가 맞아도 실제 조작감이 어색한 경우가 많았다.
- 비행 물리는 수식이 맞다고 느낌이 맞지 않는다. DragCoeff 0.01 차이가 조작감에서 크게 체감된다.
- PIE 반복 테스트. DragCoeff 0.92~0.96 사이를 0.01 단위로 조정, Z감쇠를 XY보다 약 1.5배 강하게 설정했을 때 자연스러운 호버감이 나왔다.

공식이 없는 튜닝이라 시간이 예상보다 많이 들었다. 수치 기반의 플레이테스트 기록을 남겼다면 더 체계적으로 접근할 수 있었을 것이다.

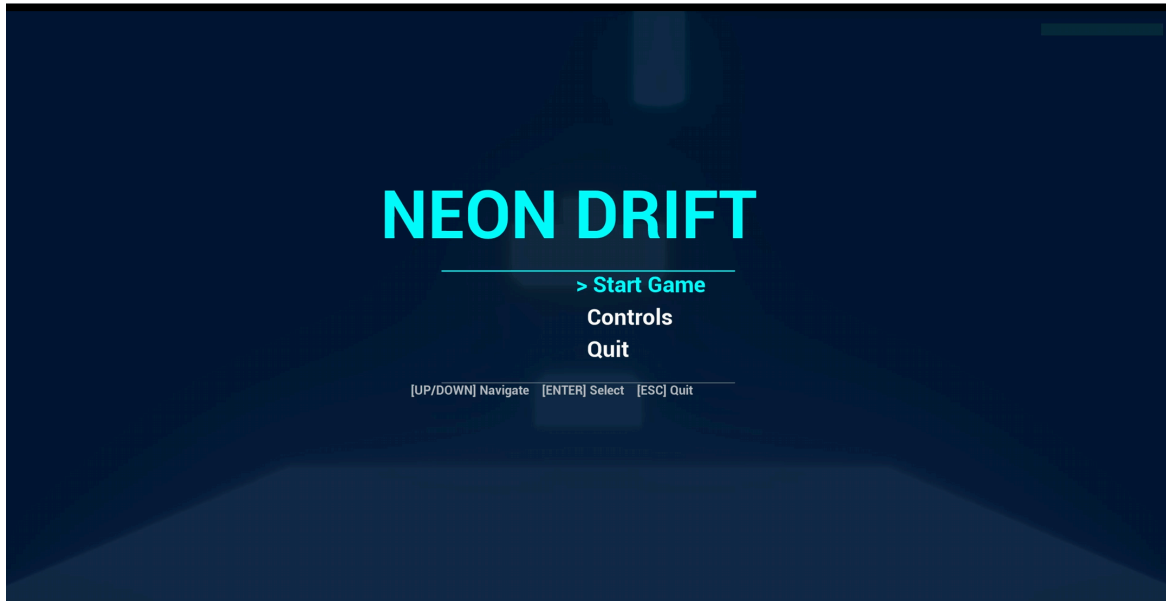
5. TWeakObjectPtr 미적용으로 인한 댕글링 위험

- ❖ AResourceShard가 호밍 대상 APlayerShip을 raw 포인터로 캐싱했다. 플레이어가 죽으면 포인터가 댕글링 상태가 된다.
- UE GC는 UPROPERTY()로 마킹된 참조만 추적한다. raw 포인터는 대상이 회수돼도 포인터가 그대로 남는다.
- TWeakObjectPtr<APlayerShip>으로 변경. 매 틱 IsValid() 체크 한 줄로 안전하게 처리.

코드 리뷰가 있었다면 초안에서 잡혔을 문제다. 1인 개발의 한계 중 하나로, 직접 테스트하다 발견했다.

[프로젝트 결과]

[메인화면 .gif]



게임 진입 시 가장 먼저 보이는 화면이다. 별도의 레벨 전환 없이 GameMode의 상태머신이 MainMenu 페이지로 시작하며, 입력을 받으면 PreWave로 전이된다.

메뉴 UI 역시 위젯 블루프린트 없이 C++로 직접 구성했다.

[비행 + 자원 채취 장면 .gif]



호버바이크로 맵을 자유롭게 비행하며 자원 블록을 파괴하고 부스러기를 자석으로 흡수하는

Gather 페이지.

속도가 빨라질수록 FOV가 70°에서 110°로 넓어져 속도감을 시각적으로 강조한다.

[터렛 움직임 .gif]

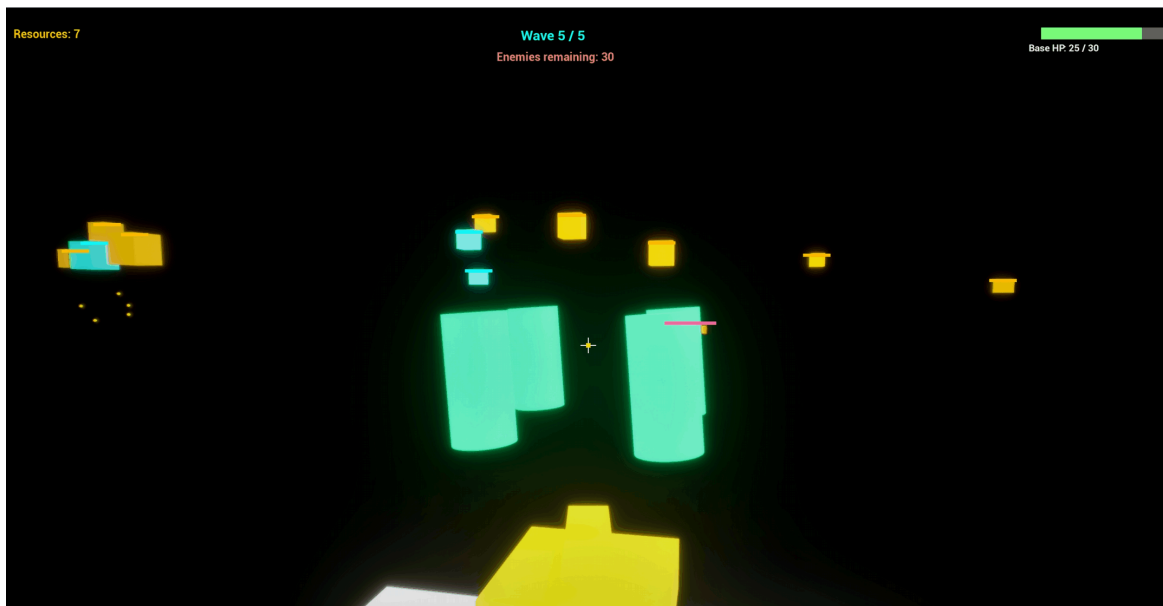


E키로 수동 터렛에 탑승하면 SetViewTargetWithBlend로 카메라가 부드럽게 전환되고, 호버바이크의 Velocity가 0으로 잠긴다. 입력은 그대로 터렛 조준으로 라우팅된다.

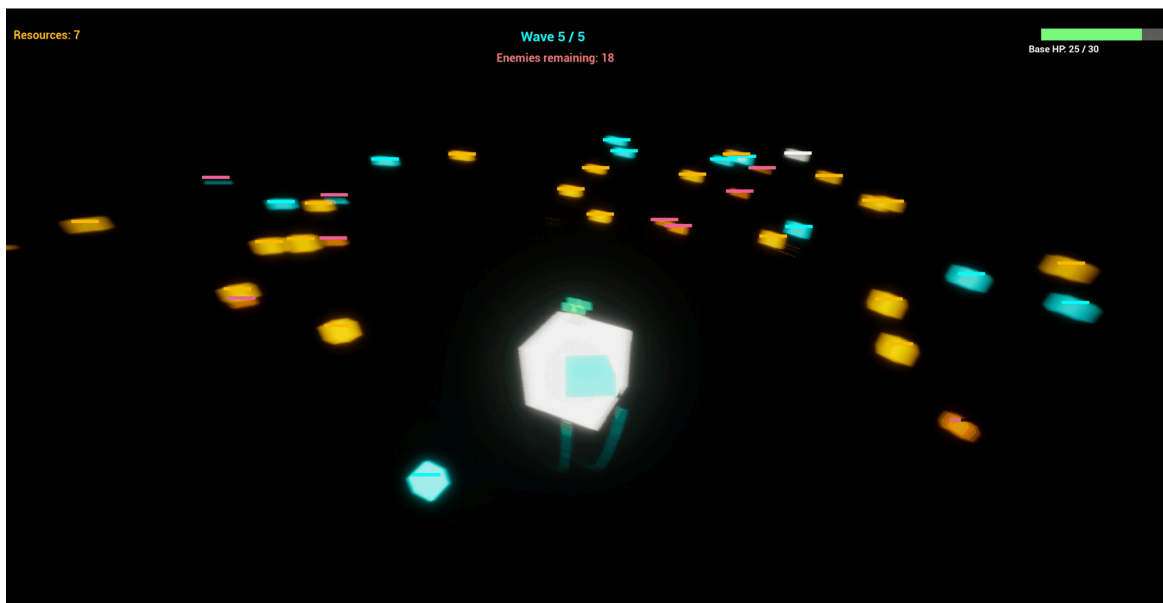
화면에 두 개의 조준점이 표시된다. 흰색 십자선은 카메라 중심, 노란 점은 포신 전방 8000uu 지점을 ProjectWorldLocationToScreen으로 투영한 실제 포신 방향이다.

포신에 회전 속도 제한이 있어 카메라를 빠르게 움직이면 둘 사이에 간격이 생기고, 이 간격 자체가 조준 지연의 시각적 피드백이 된다. 별도의 UI 없이 조준에 무게감을 주는 구조다.

[몬스터 웨이브 전투장면(터렛) .gif]



[몬스터 웨이브 전투장면(공중) .gif]



Combat 페이지. GameMode가 웨이브 정의에 따라 스폰 포인트를 활성화하며, 웨이브가 올라갈수록 진입 방향이 늘어난다. 1웨이브는 북쪽 단일 방향, 5웨이브는 N·E·S·W 4방향 동시 스폰으로 압박이 가중된다.

각 스폰 포인트는 독립적인 타이머로 몬스터를 방출하기 때문에 방향마다 다른 리듬으로 밀려온다. 수동 터렛에 탑승해 직접 조준하거나 자동 터렛의 커버 범위를 활용하는 전략적 선택이 이 페이지의 핵심이다.

[상점 .gif]



웨이브 클리어 후 자동으로 Shop 페이지로 전환된다. 방향키로 항목을 이동하고 Enter로 구매하는 방식이며, UI 전체가 위젯 블루프린트 없이 C++ UCanvasPanel로 구성되어 있다. 강화 항목은 기체4종(자석 반경·공격력·이동속도·HP)과 터렛4종(회전속도·연사속도·자동터렛 수·포탑 공격력) 총 8가지다. 구매한 강화는 UNeonGameInstance에 저장되어 웨이브가 바뀌어도, 캐릭터가 리스폰되어도 유지된다. 파생 스탯 계산도 GameInstance가 단일 진실 공급원으로 책임진다.

[프로젝트 회고]

잘 한 점	1. 설계를 먼저 끝내고 코딩을 시작했다 클래스 다이어그램과 상태머신을 코딩 전에 완전히 확정했다. 덕분에 구현 도중 구조를 바꾸는 일이 없었고, 3일이라는 짧은 기간 안에 완성할 수 있었다. 설계에 투자한 시간이 구현 속도로 돌아왔다. 2. 단일 진실 공급원 원칙을 일관되게 지켰다 GameMode 하나만 Phase 변수를 소유하고, 모든 액터가 이를 참조하는 구조를 처음부터 끝까지 지켰다. 덕분에 상태 관련 버그가 거의 발생하지 않았고, 발생해도 GameMode 한 파일만 보면 됐다. 3. IDamageable 인터페이스로 확장에 유연한 구조를 만들었다 피격 가능한 액터가 늘어도 발사 로직을 건드릴 필요가 없다. 실제로 불록·몬스터·기지를 추가할 때 인터페이스만 구현하면 됐고, 타입 캐스팅 분기가 단 한 줄도 없다. 4. 블루프린트 0% 원칙을 끝까지 유지했다 타협하고 싶은 순간이 있었다. 특히 UI 좌표 잡을 때. 그럼에도 원칙을 지킨 덕분에 코드베이스 전체가 Git diff로 추적 가능하고, AI 도구와의 연동도 막힘이 없었다.
개선 사항	1. UI는 C++과 에디터를 병행했어야 했다 블루프린트를 완전히 배제하겠다는 원칙은 지금도 맞다고 생각한다. 문제는 그 원칙을 UI 레이아웃 배치에까지 적용한 것이었다. 에디터 미리보기 없이 좌표를 잡다 보니 수치 하나 바꿀 때마다 빌드를 다시 해야 했고, 생각보다 시간이 많이 날아갔다. 로직은 코드로, 배치는 에디터로 역할을 나눴다면 원칙도 지키고 시간도 아꼈을 것이다. 원칙이 맞아도 어디까지 적용할지는 따로 생각해야 한다는 걸 배웠다.

2. 몬스터 AI가 너무 단순하다

솔직히 3일 안에 끝내야 했으니 직진 패턴으로 타협한 부분이 있다. 웨이브 숫자가 올라가도 몬스터 행동이 똑같으니 후반으로 갈수록 전략적인 재미보다 몰량 압박만 느껴진다. 처음 설계할 때 AI 타입을 인터페이스나 컴포넌트 구조로 잡아줬다면 플랭크, 돌진, 원거리 같은 타입을 나중에 끼워 넣기 쉬웠을 텐데, 그 부분까지 미리 생각하지 못했다.

3. 수치 기반 플레이테스트 기록을 남기지 않았다

DragCoeff 같은 물리 파라미터를 PIE에서 수십 번 돌려가며 맞췄는데, 그 과정을 아무것도 안 남겼다. 지금 당장은 기억하지만 시간이 지나서 다시 건드리려면 또 처음부터 감을 잡아야 한다. 튜닝 과정에서 어떤 수치가 왜 어색했는지 간단하게라도 기록해줬다면 나중에 훨씬 편했을 것이다.

4. 하드코딩된 수치가 너무 많다

웨이브별 몬스터 수, 강화 수치, 블록별 RequiredPower가 전부 코드 안에 박혀있다. 지금 규모에서는 크게 불편하지 않지만, 밸런싱을 조금만 손보려 해도 코드를 직접 열어야 한다. 데이터 테이블이나 CSV로 분리해줬다면 빌드 없이 수치만 바꿔서 바로 확인할 수 있었을 텐데, 시간에 쫓기다 보니 그 부분까지 손이 안 갔다.

5. TActorIterator를 매 틱 호출하는 부분이 있다

자석 시스템과 자동 터렛이 매 틱마다 월드 전체를 돌면서 액터를 찾는다. 지금처럼 액터 수가 적을 때는 체감이 안 되지만, 몬스터가 많이 쌓이는 후반 웨이브에서는 부담이 될 수 있다. 오버랩 이벤트로 범위 안에 들어온 것만 추적하거나, 한 번 수집한 목록을 캐싱하는 방식으로 바꿨다면 더 나았을 것이다.

일단 동작하게 만드는 데 급해서 최적화를 미뤘다.

